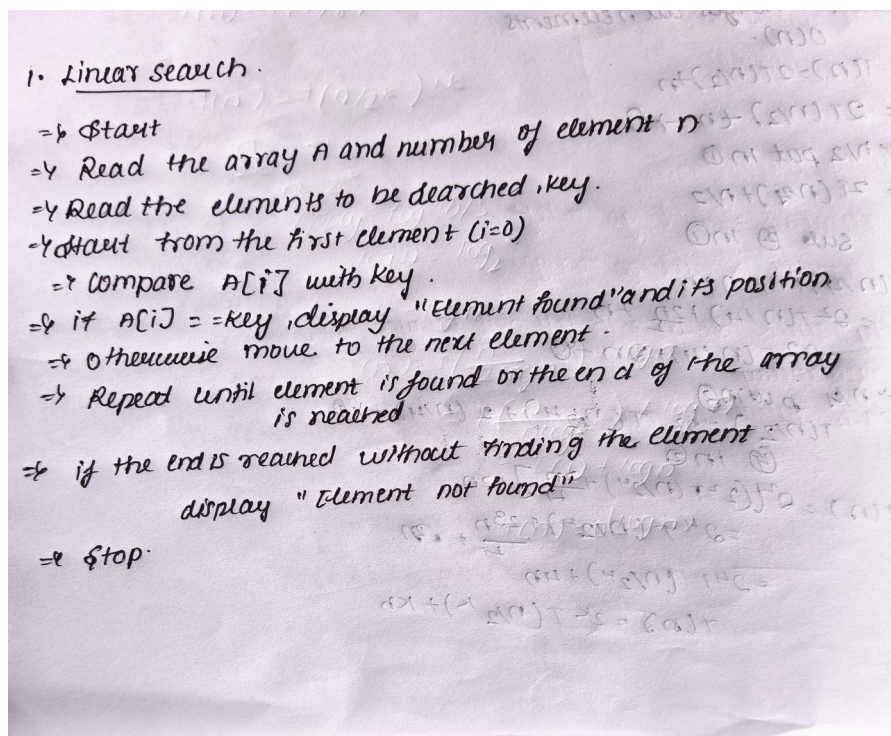


PRACTICAL: 02

Aim: To implement and find the time analysis of Linear Search and Binary Search algorithm.

A) Linear Search:

Algorithm:



Program / Code:

```
#include <iostream>

using namespace std;

int main() {
    int arr[100], i, n, key;

    cout<<"enter number of array elements:"<<endl;

    cin>>n;

    cout<<"enter array elements:"<<endl;
```



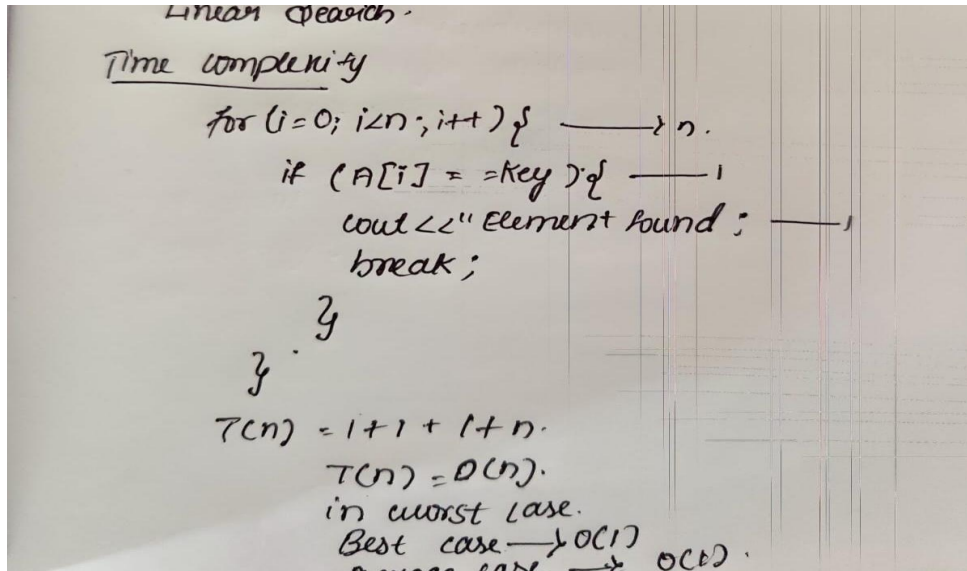
```
for(i=0;i<n;i++){  
    cin>>arr[i];  
}  
cout<<"enter element to serach:"<<endl;  
cin>>key;  
for(i=0;i<n;i++){  
    if(arr[i]==key)  
    {  
        cout<<"element found at index of:"<<i;  
    }  
}  
return 0;  
}
```

Output:

```
"C:\Users\karth\Downloads\D x + v  
enter number of array elements:  
4  
enter array elements:  
3  
6  
4  
9  
enter element to serach:  
4  
element found at index of:2  
Process returned 0 (0x0) execution time : 20.770 s  
Press any key to continue.  
|
```



Time Complexity:





B) Binary Search:

Algorithm:

Binary Search
⇒ Start
⇒ Read the sorted array A and number of elements n.
⇒ Read the element to search, key
⇒ Set low = 0
 high = n - 1
⇒ Find the middle element $mid = \frac{low + high}{2}$
* if $A[mid] == key$, element is found
* if $key < A[mid]$ search left half
 high = mid - 1
* if $key > A[mid]$ search right half
 low = mid + 1
⇒ Repeat until low > high
⇒ if low > high element is not found
⇒ Stop

Program / Code:

```
#include <iostream>

using namespace std;

void bubblesort(int arr[], int n) {
    for(int i=0; i<n-1; i++) {
        for(int j=0; j<n-i-1; j++) {
            if(arr[j] > arr[j+1]) {
                int temp = arr[j];
```



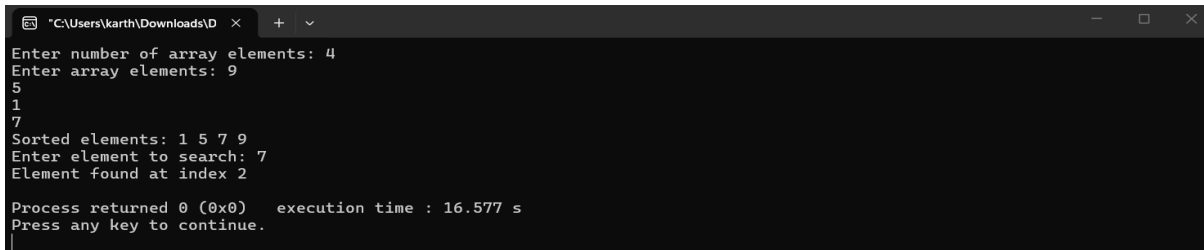
```
        arr[j] = arr[j+1];
        arr[j+1] = temp;
    }
}
}
}

int binarySearch(int arr[], int n, int key) {
    int low = 0, high = n-1;
    while(low <= high) {
        int mid = (low + high) / 2;
        if(arr[mid] == key) return mid;
        else if(arr[mid] < key) low = mid+1;
        else high = mid-1;
    }
    return -1;
}

int main() {
    int arr[100], n, key;
    cout << "Enter number of array elements: ";
    cin >> n;
    cout << "Enter array elements: ";
    for(int i=0; i<n; i++) {
        cin >> arr[i];
    }
    bubblesort(arr, n);
    cout << "Sorted elements: ";
    for(int i=0; i<n; i++) {
```

```
        cout << arr[i] << " ";  
    }  
    cout << endl;  
    cout << "Enter element to search: ";  
    cin >> key;  
    int result = binarySearch(arr, n, key);  
    if(result != -1)  
        cout << "Element found at index " << result << endl;  
    else  
        cout << "Element not found" << endl;  
    return 0;  
}
```

Output:



```
"C:\Users\karth\Downloads\D x + v  
Enter number of array elements: 4  
Enter array elements: 9  
5  
1  
7  
Sorted elements: 1 5 7 9  
Enter element to search: 7  
Element found at index 2  
  
Process returned 0 (0x0)   execution time : 16.577 s  
Press any key to continue.  
|
```

Time Complexity:

Binary Search

→ Binary search only in half $\rightarrow T(n/2)$

→ The middle element comparison takes constant time

time $\rightarrow +1$

$$T(n) = T(n/2) + 1 \rightarrow \text{①}$$

$n \rightarrow n/2$ put in ①

$$T(n/2) = T(n/4) + 1 \rightarrow \text{②}$$

$$T(n) = T(n/4) + 1 + 1$$

$$T(n) = T(n/4) + 2 \rightarrow \text{③}$$

$n \rightarrow n/4$ put in ③

$$T(n/4) = T(n/8) + 1$$

$$T(n) = T(n/8) + 3$$

$\vdots$

$$T(n) = T(n/2^k) + k$$

$\frac{n}{2^k} = 1$

$n = 2^k$

Apply log b/s

$$\log n = \log_2 k$$

$$k = \log n$$

$$T(n) = T\left(\frac{n}{2^{\log n}}\right) + \log n$$

$$= T\left(\frac{n}{n^{\log n}}\right) + \log n$$

$$= T(1) + \log n$$

$$T(n) = O(\log n)$$

$T(n) = O(\log n) \rightarrow$ in worst case

$O(1) \rightarrow$ in Best case

$O(\log n) \rightarrow$ in Average case

Conclusion:

In this practical, we studied and implemented two searching algorithms: Linear Search checks each element one by one, so it takes more time as the list size grows. Binary Search is much faster because it works only on sorted lists and cuts the list in half every time, making it a better choice for large data.